

Openmower-Taf-WUP - Dokumentation - JSON Flow Configuration

- v1.2

Angabe	Wert
Projektname	Openmower-Taf-WUP
Dokumentenart	Dokumentation
Thema	JSON Flow Configuration
Version	v1.2
Status	Freigegeben
Erstellungsdatum	2026-07-07
Aenderungsdatum	2026-07-07
Autor / Bearbeiter	ChatGPT
Ablageort	Projektpaket: docs/
Referenzen	README.md, docs/json-flow-configuration.md, bridge/controller.py, controller_data/bot_commands.json

1. Zusammenfassung

Die Mobert Flow-Konfiguration wurde komplett von XML auf JSON umgestellt. Die aktive Laufzeitdatei ist jetzt /data/bot_commands.json. XML-Parsing und die alte legacy mobertCommands-Unterstützung wurden aus dem Controller entfernt.

Die bisherige Logik mit zentralen Modulen, Watchdogs, Outputs, WhatsApp-Befehlen, MQTT-Triggern und Statusfunktionen bleibt strukturell erhalten. Die Datei ist jetzt fuer eine zweite App leichter lesbar und schreibbar.

2. Versionierung

Bereich	Alter Stand	Neuer Stand
Auslieferung / Dokumentation	v1.1	v1.2
Bot-Konfigurationsformat	mobertBotConfig XML 0.4	mobertBotConfig JSON 0.5
Aktive Datei	/data/bot_commands.xml	/data/bot_commands.json
Beispieldatei im Image	/app/bot_commands.example.xml	/app/bot_commands.example.json

3. Wichtige Aenderungen

- bot_commands.xml wurde durch bot_commands.json ersetzt.
- Der Controller akzeptiert keine XML-Konfiguration mehr.
- bridge/bot_commands.example.json ist der neue Fallback im Docker-Image.
- controller_data/bot_commands.json ist die aktive auslieferbare Volume-Konfiguration.
- messenger/bot/commands/set/config/json ersetzt das fruehere XML-Set-Topic.
- messenger/bot/commands/source/json veroeffentlicht die aktive JSON-Quelle.
- messenger/bot/commands/json bleibt die geparte Command-/Flow-Uebersicht.
- Flow-head.show wurde ergaenzt: show=false blendet einen Flow aus der Hilfe aus, ohne ihn zu deaktivieren.

4. JSON-Grundstruktur

Die neue Datei folgt dieser Struktur:

```
{
  "format": "mobertBotConfig",
  "version": "0.5",
  "language": "de",
  "head": { ... },
  "modules": { ... },
}
```

```
"flows": { ... }
}
```

5. Module

Modul	Aufgabe
whatsapp	Zentrale WAHA-Session, Default-Gruppe, Lauschgruppe und Wake Word.
whatsapp_watchdog	WhatsApp-Eingang fuer Mober-Befehle, verweist auf whatsapp.
whatsapp_output	WhatsApp-Ausgabe fuer Antworten und Meldungen, verweist auf whatsapp.
mqtt_watchdog	MQTT-Eingang fuer automatische Flow-Trigger und Bestaetigungen.
mqtt_output	MQTT-Ausgabe fuer OpenMower-Aktionen und Steuerbefehle.

6. Flow-Aufbau

Jeder Flow besitzt einen head-Block und mindestens einen Step. enabled steuert die Ausfuehrung. show steuert nur die Anzeige in Mober: ? und messenger/bot/help/json.

JSON-Pfad	Bedeutung
flows.<flow_id>.head.name	Anzeigenname des Flows.
flows.<flow_id>.head.description	Beschreibung fuer Hilfe und Dokumentation.
flows.<flow_id>.head.enabled	Aktiviert oder deaktiviert den Flow.
flows.<flow_id>.head.show	Blendet den Flow in der Hilfe ein oder aus.
flows.<flow_id>.steps.<step_id>.input.expect.command	WhatsApp-Befehl, z. B. Status.
flows.<flow_id>.steps.<step_id>.processing.mode	Lokale Verarbeitung, MQTT-Passthrough oder Statusfunktion.
flows.<flow_id>.steps.<step_id>.outputs	WhatsApp- oder MQTT-Ausgaben.

7. MQTT-Schnittstelle

Topic	Funktion
messenger/bot/commands/source/json	Aktive JSON-Konfiguration als Quelle.
messenger/bot/commands/json	Geparste Command-/Flow-Uebersicht.
messenger/bot/commands/version	Geladene JSON-Konfigurationsversion.
messenger/bot/commands/set/config/json	Neue JSON-Konfiguration per MQTT speichern und laden.
messenger/bot/commands/set/renew/json	Bestehende JSON-Datei vom Datentraeger neu laden.
messenger/bot/commands/validation/json	Validierungsstatus nach Laden oder Setzen.
messenger/bot/help/text	Generierte WhatsApp-Hilfe als Text.
messenger/bot/help/json	Generierte Hilfe mit Metadaten.

8. Deployment-Hinweise

- Bestehende Volumes muessen auf controller_data/bot_commands.json umgestellt werden.
- BOT_COMMANDS_FILE sollte auf /data/bot_commands.json zeigen.
- DEFAULT_BOT_COMMANDS_FILE sollte auf /app/bot_commands.example.json zeigen.
- Nach dem Austausch den Controller neu bauen oder die Datei per MQTT set/config/json setzen.
- Alte retained Werte auf messenger/bot/commands/xml werden beim Veroeffentlichen der Bot-Kommandos geleert.

Beispiel Reload: mosquitto_pub -h Mosquitto -t messenger/bot/commands/set/renew/json -m '{}'

Beispiel Set: mosquitto_pub -h Mosquitto -t messenger/bot/commands/set/config/json -f bot_commands.json

9. Geaenderte Dateien

Datei	Aenderung
.env.example	Default-Pfad auf bot_commands.json umgestellt.
bridge/Dockerfile	Kopiert bot_commands.example.json ins Image.
bridge/controller.py	JSON-Loader, JSON-MQTT-Set, source/json, show-Unterstützung, XML entfernt.
bridge/bot_commands.example.json	Neue Beispielkonfiguration.
controller_data/bot_commands.json	Neue aktive Konfiguration fuer /data.
compose.example.yaml	Default-Pfad auf bot_commands.json umgestellt.
README.md und docs/*.md	Dokumentation auf JSON-only umgestellt.
CHANGELOG.md	v1.2 Eintrag ergaenzt.

10. Pruefung

Pruefung	Ergebnis
Python Syntax	bridge/controller.py wurde mit py_compile geprueft.
JSON Validierung	bridge/bot_commands.example.json und controller_data/bot_commands.json wurden geladen und auf Format, Version und Flow-Struktur geprueft.
Controller Loader	Der Controller-Loader wurde mit einem MQTT-Mock importiert; 20 WhatsApp-Befehle und 24 Flows wurden aus JSON geladen.

11. Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v1.2	2026-07-07	ChatGPT	Freigegeben	Komplette Umstellung von XML auf JSON dokumentiert.